

MODÜL 07

Transfer Learning

Feature Extraction · Fine-tuning · Domain Adaptation

VGG16 · ResNet50 · Freeze/Unfreeze · Diferansiyel LR · TF · PyTorch

Seviye	Orta-İleri — Uygulamalı
Kapsam	Özellik Hiyerarşisi · 3 Strateji · Freeze/Unfreeze · Diferansiyel LR
Framework	NumPy (kavramsal) · TensorFlow/Keras · PyTorch
Önceki Modül	Modül 06 — CNN
Sonraki Modül	Modül 08 — RNN / LSTM / GRU

1. Öğrenme Hedefleri

- ✓ Özellik hiyerarşisini kullanarak transfer learning'in neden işe yaradığını açıklamak
- ✓ Feature extraction, fine-tuning ve full training stratejilerini karşılaştırmak
- ✓ Hangi durumda hangi stratejiyi seçeceğine karar vermek
- ✓ Freeze/unfreeze katman stratejisini doğru uygulamak
- ✓ Diferansiyel learning rate kavramını açıklamak ve uygulamak
- ✓ TF/Keras ve PyTorch'ta pretrained model kullanımını kodlamak
- ✓ Domain adaptation kavramını açıklamak

2. Transfer Learning Neden İşe Yarar?

Derin CNN'ler eğitilirken katmanlara göre farklı soyutlama seviyeleri öğrenir. Alt katmanlar evrensel düşük seviyeli özellikler öğrenirken (kenarlar, köşeler), üst katmanlar veri setine özgü yüksek seviyeli özellikler öğrenir. Bu hiyerarşi, öğrenilen düşük/orta seviye özelliklerin farklı görevlere transfer edilmesini mümkün kılar.

Katman Grubu	Öğrenilen Özellikler	Transfer Edilebilirlik	Strateji
Katman 1-3	Kenarlar, köşeler, renkler	%95 — çok yüksek	Her zaman dondur
Katman 4-8	Parçalar, dokular, şekiller	%75 — yüksek	Genelde dondur
Katman 9-12	Nesneler, anlamsal yapı	%40 — orta	Fine-tune ile aç
FC Katman	Sınıflandırıcı	%5 — çok düşük	Her zaman yeniden eğit

Pratik sonuç: ImageNet'te ResNet50 eğitmek ~1 ay (V100 GPU). Pretrained model + fine-tuning ile aynı kalite sadece birkaç saatte ulaşılabilir. Az veri durumunda pretrained olmadan yeterli genelleme neredeyse imkansızdır.

3. Üç Transfer Stratejisi

3.1 Feature Extraction (Özellik Çıkarma)

Tüm backbone katmanları dondurulur. Yalnızca yeni sınıflandırıcı katman (head) eğitilir. Backbone sabit bir özellik çıkarıcı gibi davranır.

Avantaj: En hızlı eğitim, az GPU belleği

Avantaj: Az veriye çok uygun, overfitting riski düşük

Dezavantaj: Hedef alan kaynaktan çok farklıysa yetersiz

LR: head: $1e-3$

Ne zaman: Az veri ($<1K$) + benzer domain

3.2 Fine-tuning

Alt katmanlar dondurulur, üst katmanlar (son 2-4 blok) + head küçük learning rate ile eğitilir. Backbone özellikleri hedef alana adapte olur.

Avantaj: Feature extraction'dan daha iyi genelleme

Avantaj: Orta-az veriyle çalışır

Dezavantaj: Daha uzun eğitim, dikkatli lr seçimi

LR: backbone: $1e-5$, head: $1e-3$ (diferansiyel)

Ne zaman: Orta veri ($1K-100K$) + benzer/farklı domain

3.3 Full Training

Hiçbir katman dondurulmaz. Tüm ağırlıklar baştan eğitilir. Pretrained ağırlıklar yalnızca başlangıç noktası olarak kullanılır.

Avantaj: Maksimum esneklik, hedef alana tam uyum

Dezavantaj: Çok fazla veri ve GPU süresi gerekir

Dezavantaj: Az veriyle kolayca overfitting

LR: tüm katmanlar: $1e-3$ veya lr scheduler ile

Ne zaman: Çok veri ($>100K$) + kaynaktan çok farklı domain

3.4 Strateji Seçim Tablosu

Veri Miktarı	Domain Benzerliği	Önerilen Strateji	Learning Rate
Az ($<1K$)	Benzer	Feature Extraction	$1e-3$ (head)
Az ($<1K$)	Farklı	Feature Ext. + daha fazla veri toplama	$1e-3$
Orta ($1K-10K$)	Benzer	Fine-tune (son 2 blok)	$1e-5/1e-3$
Orta ($1K-10K$)	Farklı	Fine-tune (derin)	$1e-5/1e-3$
Çok ($>100K$)	Benzer	Fine-tune (tüm bloklar)	$1e-5/1e-4$
Çok ($>100K$)	Farklı	Full Training	$1e-3$

4. Freeze/Unfreeze Stratejisi

Freeze: parametre requires_grad=False → gradyan hesaplanmaz → güncelleme olmaz.
Bu sayede pretrained özellikler korunur.

TensorFlow / Keras:

```
base_model.trainable = False # Tüm backbone dondur
# veya seçici:
for layer in base_model.layers[:-30]:
    layer.trainable = False # Son 30 hariç dondur
```

PyTorch:

```
for param in model.parameters(): # Tüm backbone dondur
    param.requires_grad = False
# veya seçici blok:
for param in model.layer4.parameters():
    param.requires_grad = True # layer4'ü aç
```

5. Diferansiyel Learning Rate

Fine-tuning'de farklı katman gruplarına farklı öğrenme hızı uygulamak kritik öneme sahiptir. Alt katmanlar iyi özellikler içerir — büyük lr ile bunları bozmamalıyız. Yeni eklenen head hızlı öğrenmeli.

Kural: lr katman derinliğiyle azalır (alt katmanlara doğru)

Katman Grubu	Learning Rate	Neden?
Layer 1-3	1e-6 veya dondurulmuş	Evrensel özellikler — değiştirme
Layer 4-6	1e-5	Genel özellikler — çok az değiştir
Layer 7-9	1e-5	Orta özellikler — az değiştir
Layer 10-12	1e-4	Göreve yakın — biraz değiştir
Yeni Head	1e-3	Sıfırdan öğrenecek — büyük adım

PyTorch Diferansiyel LR Kodu:

```
optimizer = torch.optim.Adam([
    {'params': model.layer1.parameters(), 'lr': 1e-5},
    {'params': model.layer2.parameters(), 'lr': 1e-5},
    {'params': model.layer3.parameters(), 'lr': 1e-4},
    {'params': model.layer4.parameters(), 'lr': 1e-4},
    {'params': model.fc.parameters(), 'lr': 1e-3},
])
```

6. Domain Adaptation

Domain adaptation: Kaynak ve hedef domain farklı dağılımlara sahip olduğunda modelin hedef domain'e uyarlanması. Örneğin: ImageNet (renkli fotoğraf) → X-ray (gri tonlamalı tıbbi görüntü).

Senaryo	Strateji	Not
Renkli → Gri tonlamalı	Fine-tune	İlk Conv için 1-kanal giriş
Doğal görüntü → Tıbbi görüntü	Derin fine-tune	Farklı özellik yapısı
Fotoğraf → Çizim/Animasyon	Fine-tune veya full	Yüksek domain farkı
Büyük varlık → Küçük varlık	Feature ext.	Benzer özellikler yeterli
ImageNet → Uydu görüntüsü	Fine-tune (orta derin)	Spatial özellikler benzer

7. Yaygın Hatalar

Hata 1: include_top=True ile pretrained model yüklemek	include_top=True modelin orijinal FC katmanını dahil eder (örn. 1000 sınıf). Custom görev için include_top=False kullan.
Hata 2: Fine-tuning'de çok büyük lr kullanmak	Pretrained ağırlıklar büyük lr ile bozulur — catastrophic forgetting. Fine-tuning için 1e-5 veya daha küçük başla.
Hata 3: Backbone'u dondurmadan feature extraction yapmak	Dondurmazsak backbone da güncellenir — bu feature extraction değil, full training olur. layer.trainable=False veya requires_grad=False
Hata 4: model.eval() / model.train() modunu unutmak	Fine-tuning eğitiminde model.train() gerekli — BN ve Dropout doğru çalışsın. Inference'da model.eval()
Hata 5: Giriş normalizasyonu yapmamak	Pretrained modeller belirli normalizasyon bekler (ImageNet: mean=[0.485,0.456,0.406], std=[0.229,0.224,0.225]). Eksik normalizasyon performansı düşürür.

8. Özet Tablo

Kavram	Açıklama	Pratik Kullanım
Feature Extraction	Backbone dondur, sadece head eğit	Az veri, hızlı prototip
Fine-tuning	Üst katmanlar + head küçük lr ile eğit	Orta veri, iyi sonuç
Full Training	Hiçbir şey dondurulmaz	Çok veri, max esneklik
Freeze	requires_grad=False, güncelleme yok	Pretrained koruma
Diferansiyel LR	Katman grubuna göre farklı lr	Fine-tuning kalitesi
Domain Adaptation	Kaynak→hedef domain uyarlaması	Farklı görev tipleri
include_top=False	FC katman dahil etme, sadece backbone	Custom sınıflandırma

9. Kaynaklar

- [1] Yosinski, J. et al. (2014). How Transferable are Features in Deep Neural Networks? NeurIPS.
- [2] Donahue, J. et al. (2014). DeCAF: A Deep Convolutional Activation Feature for Generic Visual Recognition. ICML.
- [3] Howard, J. & Ruder, S. (2018). Universal Language Model Fine-Tuning for Text Classification (ULMFiT). ACL.
- [4] He, K. et al. (2016). Deep Residual Learning for Image Recognition. CVPR. (ResNet transfer base)
- [5] TensorFlow Transfer Learning Guide.
https://www.tensorflow.org/tutorials/images/transfer_learning
- [6] PyTorch Transfer Learning Tutorial.
https://pytorch.org/tutorials/beginner/transfer_learning_tutorial.html
- [7] Tan, C. et al. (2018). A Survey on Deep Transfer Learning. arXiv:1808.01974.

Sonraki Adım: Modül 08 — RNN / LSTM / GRU | Zaman Serisi • BPTT • LSTM Kapıları • Seq2Seq